

```

64 var x = o.left;
65 var y = o.top;
66
67 var ax = settings.accX;
68 var ay = settings.accY;
69 var th = t.height();
70 var wh = w.height();
71 var tw = t.width();
72 var ww = w.width();
73
74 if (y + th + ay >= b &&
75     y <= b + wh + ay &&
76     x + tw + ax >= a &&
77     x <= a + ww + ax) {
78     //trigger the custom event
79     if (!t.appeared) t.trigger('appear', settings.data);
80 } else {
81     //it scrolled out of view
82     t.appeared = false;
83 }
84 };
85
86 //create a modified fn with some additional logic
87 var modifiedFn = function() {
88
89     //mark the element as visible
90     t.appeared = true;
91
92     //is this supposed to happen only once?
93     if (settings.one) {
94
95         //remove the check
96         w.unbind('scroll', check);
97         var i = $.inArray(check, $.fn.appear.checks);
98         if (i >= 0) $.fn.appear.checks.splice(i, 1);
99     }
100
101     //trigger the original fn
102     fn.apply(this, arguments);
103
104     //bind the modified fn to the element
105     //as one) t.one('appear', settings.data, modifiedFn);
106     settings.data, modifiedFn);
107
108 };

```



MATERIA:

Programación Orientada a Objetos

Unidad 4 Persistencia, Archivos y Concurrencia Moderna

Docente:

Grupo:

Unidad #4 - Persistencia, Archivos y Concurrencia Moderna.

4.2. Modelos básicos de persistencia (Archivos planos).

4.2.1. Información sobre archivos y directorios. La clase File.

4.2.2. Lectura y escritura de archivos de texto: FileReader-BufferedReader, FileWriter-PrintWriter.

4.2.3. Lectura y escritura de datos primitivos en archivos: FileOutputStream-DataOutputStream, FileInputStream-DataInputStream

4.2.4. Manejo de archivos JSON/XML con Jackson y Gson

4.2. Modelos básicos de persistencia (Archivos planos).

¿Qué es la Persistencia?

Capacidad de almacenar datos de forma permanente, más allá de la ejecución del programa.

¿Por qué es importante?

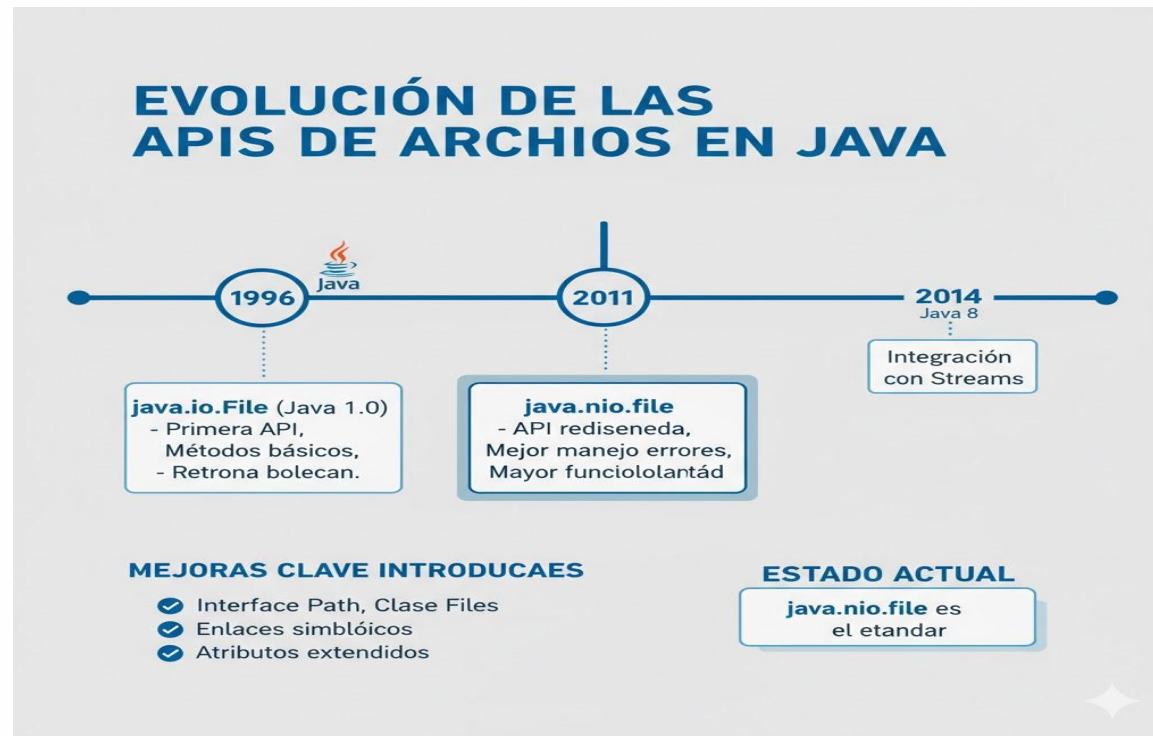
- Los datos en memoria se pierden al cerrar el programa
- Necesidad de guardar información entre ejecuciones
- Compartir datos entre diferentes aplicaciones
- Crear respaldos y recuperación ante fallos

Tipos de persistencia

- **Archivos planos** - Texto o binario simple
- **Bases de datos** - Estructurado y relacional
- **Servicios en la nube** - Almacenamiento remoto
- **Serialización** - Objetos completos

4.2. Modelos básicos de persistencia (Archivos planos).

EVOLUCIÓN DE LAS APIs EN JAVA



CREAR OBJETOS PATH

```
java
// Java 11+ (recomendado)
Path ruta1 = Path.of("datos", "archivo.txt");
Path ruta2 = Path.of("datos/archivo.txt");

// Java 7-10 (compatibilidad)
Path ruta3 = Paths.get("datos", "archivo.txt");

// Ruta absoluta
Path absoluta = Path.of("/home/usuario/datos.txt");

// Obtener directorio actual
Path actual = Path.of(".").toAbsolutePath();

// Ruta desde URI
Path desdeUri = Path.of(URI.create("file:///datos/archivo.txt"));
```

Nota importante: Path.of() solo crea la referencia, NO crea el archivo físico.

MÉTODOS DE LA INTERFAZ PATH

```
java
Path archivo = Path.of("proyecto/datos/estudiantes.txt");

// Obtener componentes
archivo.getFileName(); // estudiantes.txt
archivo.getParent();    // proyecto/datos
archivo.getRoot();       // null (ruta relativa)

// Conversiones
archivo.toAbsolutePath(); // Ruta absoluta completa
archivo.normalize();      // Elimina . y ..
archivo.toRealPath();     // Resuelve enlaces simbólicos

// Comparaciones
Path otro = Path.of("proyecto/datos/notas.txt");
archivo.equals(otro);     // false
archivo.startsWith("proyecto"); // true
archivo.endsWith("estudiantes.txt"); // true

// Navegación
archivo.resolve("backup"); // proyecto/datos/backup
archivo.resolveSibling("notas.txt"); // proyecto/datos/notas.txt
```